

ALFIDO TECH

Java Developer Internship

Task 2: Student Management System ***(Console-Based Mini Project)***

Submitted By:

Alfishan Khan

B.Tech – Computer Science and Engineering

(Artificial Intelligence & Machine Learning)

Institute of Technology and Management

1. OBJECTIVE

The primary objectives of this project are:

- To understand and implement fundamental Java programming concepts.
 - To develop a menu-driven console application using Java.
 - To perform operations such as adding, viewing, and searching student records.
 - To implement arrays for storing and managing student information.
 - To use loops and conditional statements for controlling program execution.
 - To accept and process user input using the Scanner class.
 - To improve logical thinking, programming skills, and problem-solving abilities through practical implementation.
-

2. ABSTRACT

The Student Management System is a Console-Based Mini Project developed using Java as part of the Java Developer Internship at Alfido Tech. The project aims to demonstrate the practical implementation of Java fundamentals through a menu-driven application that manages student records efficiently. The application allows users to add student details, view all stored records, search for a specific student using the Student ID, and exit the application safely.

The system uses arrays for temporary data storage and employs the Scanner class for accepting user input. Various programming concepts such as loops, conditional statements, switch statements, arrays, and user input handling have been implemented throughout the project. The project provides hands-on experience in developing console-based applications and strengthens problem-solving, logical thinking, and programming skills. It serves as an effective demonstration of how fundamental Java concepts can be integrated to create a simple and functional management system.

3. INTRODUCTION

A Student Management System is an application used to manage and organize student information in a systematic manner. In educational institutions, maintaining student records manually can be time-consuming and prone to errors. Computerized systems provide an efficient way to store, retrieve, and manage such information accurately.

This project is a Console-Based Mini Project developed using Java programming language. The application follows a menu-driven approach and enables users to perform basic operations such as adding student details, viewing all stored records, searching for a student using the Student ID, and exiting the application.

The project demonstrates the practical implementation of core Java programming concepts including arrays, loops, conditional statements, switch statements, and user input handling using the Scanner class. Developing this application provided practical exposure to console-based programming and improved programming logic and analytical thinking abilities.

4. PROJECT OVERVIEW

The Student Management System provides the following functionalities:

- Add Student – Stores Student ID and Student Name.
- View Students – Displays all stored student records.
- Search Student – Searches for a student using Student ID.
- Exit – Terminates the application safely.

The application provides a simple and interactive menu-driven interface for managing student information efficiently.

5. TECHNOLOGIES USED

- Programming Language: Java
 - Development Environment: Visual Studio Code (VS Code)
 - Concepts Used:
 - Arrays
 - Scanner Class
 - Loops (do-while and for loop)
 - Conditional Statements
 - Switch-Case Statements
 - Console Input and Output
-

6. IMPLEMENTATION DETAILS

Step 1: Import the Scanner class and create the main class.

Step 2: Create arrays to store Student IDs and Student Names.

Step 3: Declare variables to maintain the student count and user choices.

Step 4: Display menu options using `System.out.println()` statements.

Step 5: Accept user input using the Scanner class.

Step 6: Implement Add Student functionality to store student information.

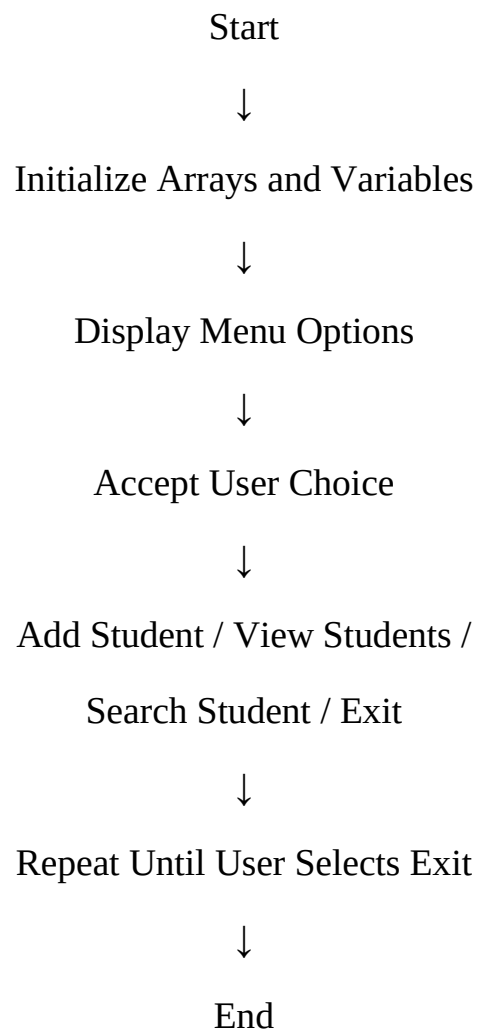
Step 7: Implement View Students functionality to display all stored records.

Step 8: Implement Search Student functionality to locate a student using the Student ID.

Step 9: Implement Exit functionality to terminate the application safely.

7. PROGRAM WORKFLOW

The workflow of the Student Management System is as follows:



8. CODE SNIPPETS

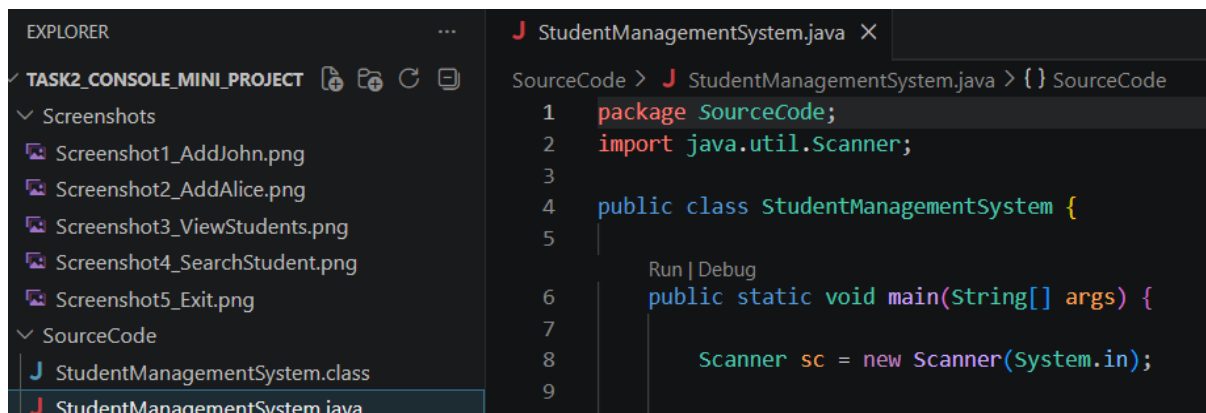


Figure 1: Import Statement, Class Declaration, and Scanner Initialization.

Explanation: This code imports the Scanner class, declares the StudentManagementSystem class, and creates a Scanner object to accept user input from the keyboard.

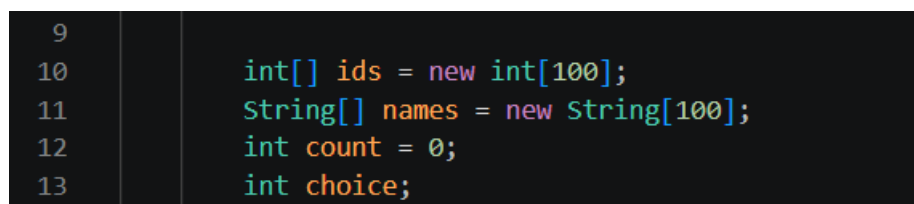


Figure 2: Declaration of Arrays and Variables

Explanation: This code declares two arrays, ids and names, to store student IDs and names respectively. It also initializes the count variable to track the number of stored records and declares the choice variable for managing menu selections in the application.

```

14
15         do {
16             System.out.println(x: "\n===== Student Management System =====");
17             System.out.println(x: "1. Add Student");
18             System.out.println(x: "2. View Students");
19             System.out.println(x: "3. Search Student");
20             System.out.println(x: "4. Exit");
21             System.out.print(s: "Enter your choice: ");
22
23             choice = sc.nextInt();
24
25             switch (choice) {
26

```

Figure 3: Menu-Driven Interface Using do-while Loop and Switch Statement

Explanation: This code displays the main menu of the Student Management System and prompts the user to select an option. A do-while loop ensures that the menu is displayed repeatedly until the user chooses to exit. The entered choice is processed using a switch statement to perform different operations such as adding, viewing, searching, and exiting.

```

27         case 1:
28             System.out.print(s: "Enter Student ID: ");
29             ids[count] = sc.nextInt();
30
31             sc.nextLine();
32
33             System.out.print(s: "Enter Student Name: ");
34             names[count] = sc.nextLine();
35
36             count++;
37             System.out.println(x: "Student Added Successfully!");
38             break;
39
40         case 2:
41             if (count == 0) {
42                 System.out.println(x: "No Students Available.");
43             } else {
44                 System.out.println(x: "\nStudent List:");
45                 for (int i = 0; i < count; i++) {
46                     System.out.println("ID: " + ids[i]
47                                     + " Name: " + names[i]);
48                 }
49             }
50             break;
51

```

Figure 4: Code for Adding and Viewing Student Records

Explanation: This code implements the functionalities for adding and viewing student records. In Case 1, the program accepts the student's ID and name from the user, stores them in arrays, increments the record count, and displays a success message. In Case 2, the program checks whether records exist and, if available, uses a for loop to display all stored student details.

```

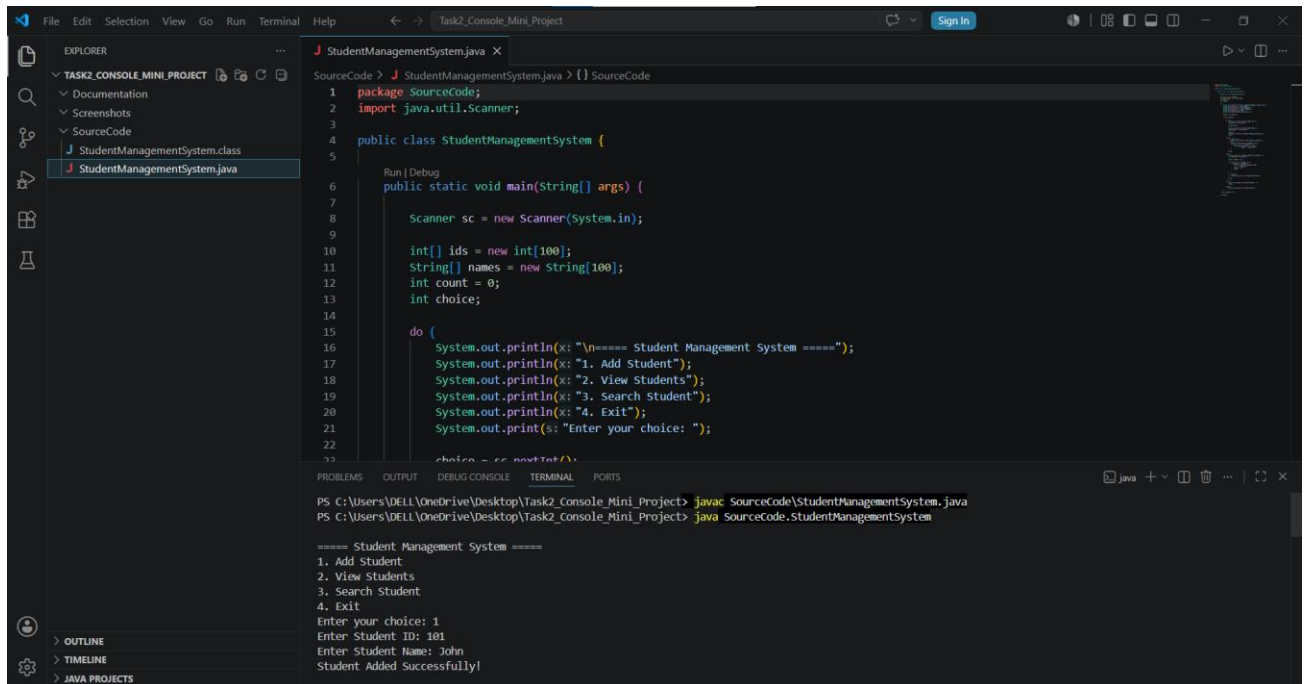
52         case 3:
53             System.out.print(s: "Enter Student ID to Search: ");
54             int searchId = sc.nextInt();
55
56             boolean found = false;
57
58             for (int i = 0; i < count; i++) {
59                 if (ids[i] == searchId) {
60                     System.out.println("Student Found: "
61                                     + names[i]);
62                     found = true;
63                     break;
64                 }
65             }
66
67             if (!found) {
68                 System.out.println(x: "Student Not Found!");
69             }
70             break;
71
72         case 4:
73             System.out.println(x: "Exiting Program...");
74             break;
75
76         default:
77             System.out.println(x: "Invalid Choice!");
78     }
79
80     } while (choice != 4);
81
82     sc.close();
83 }
84 }

```

Figure 5: Code for Searching Student Records and Exiting the Application

Explanation: This code implements the search and exit functionalities of the Student Management System. In Case 3, the program accepts a Student ID, searches through the stored records using a for loop, and displays the corresponding student's name if found; otherwise, it displays a "Student Not Found" message. In Case 4, the program terminates the application by displaying an exit message. The default case handles invalid menu selections, and the Scanner object is closed after the program execution is completed.

9. OUTPUT SCREENSHOTS

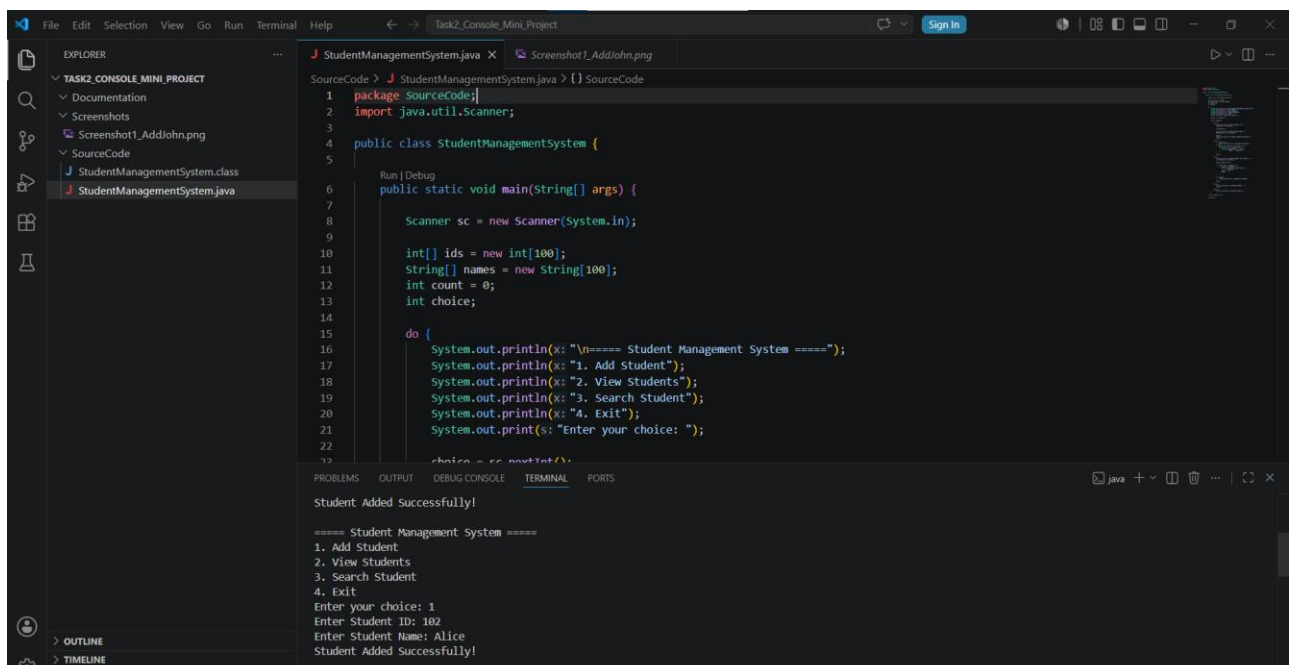


The screenshot shows an IDE with the following components:

- EXPLORER:** Shows the project structure with 'TASK2_CONSOLE_MINI_PROJECT' containing 'Documentation', 'Screenshots', 'SourceCode', 'StudentManagementSystem.class', and 'StudentManagementSystem.java'.
- SourceCode:** Displays the code for 'StudentManagementSystem.java'. The code includes a package declaration, imports, and a main method that uses a Scanner to take input and store it in arrays.
- TERMINAL:** Shows the command 'javac SourceCode\StudentManagementSystem.java' and 'java SourceCode.StudentManagementSystem'. The output displays the menu, the user's choice (1), the entered ID (101) and name (John), and the success message 'Student Added Successfully!'.

Figure 6: Adding the First Student Record

Explanation: This output demonstrates the successful addition of the first student record. The user enters the Student ID (101) and Student Name (John), and the program stores the information in arrays and displays the message "Student Added Successfully!".



The screenshot shows the same IDE as Figure 6, but with the following differences:

- EXPLORER:** A new file 'Screenshot1_AddJohn.png' has been added to the 'Screenshots' folder.
- SourceCode:** The same code as in Figure 6 is shown.
- TERMINAL:** The output shows the user entering choice 1, ID 102, and name Alice, followed by the success message 'Student Added Successfully!'.

Figure 7: Adding the Second Student Record

Explanation: This output shows the successful addition of the second student record. The user enters the Student ID (102) and Student Name (Alice), and the system stores the record and displays the message "Student Added Successfully!".

```
SourceCode > StudentManagementSystem.java > {} SourceCode
1 package SourceCode;
2 import java.util.Scanner;
3
4 public class StudentManagementSystem {
5
6     Run | Debug
7     public static void main(String[] args) {
8
9         Scanner sc = new Scanner(System.in);
10
11         int[] ids = new int[100];
12         String[] names = new String[100];
13         int count = 0;
14         int choice;
15
16         do {
17             System.out.println(x: "\n==== Student Management System =====");
18             System.out.println(x: "1. Add Student");
19             System.out.println(x: "2. View Students");
20             System.out.println(x: "3. Search Student");
21             System.out.println(x: "4. Exit");
22             System.out.print(s: "Enter your choice: ");
23
24             choice = sc.nextInt();
25
26         } while (choice != 0);
27
28     }
29 }
```

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
===== Student Management System =====
1. Add Student
2. View Students
3. Search Student
4. Exit
Enter your choice: 2

Student List:
ID: 101 Name: John
ID: 102 Name: Alice
```

Figure 8: Viewing Student Records

Explanation: This output shows the View Students functionality of the Student Management System. The system displays the details of all stored students, including their IDs and names (John and Alice), confirming that the records have been stored and retrieved successfully.

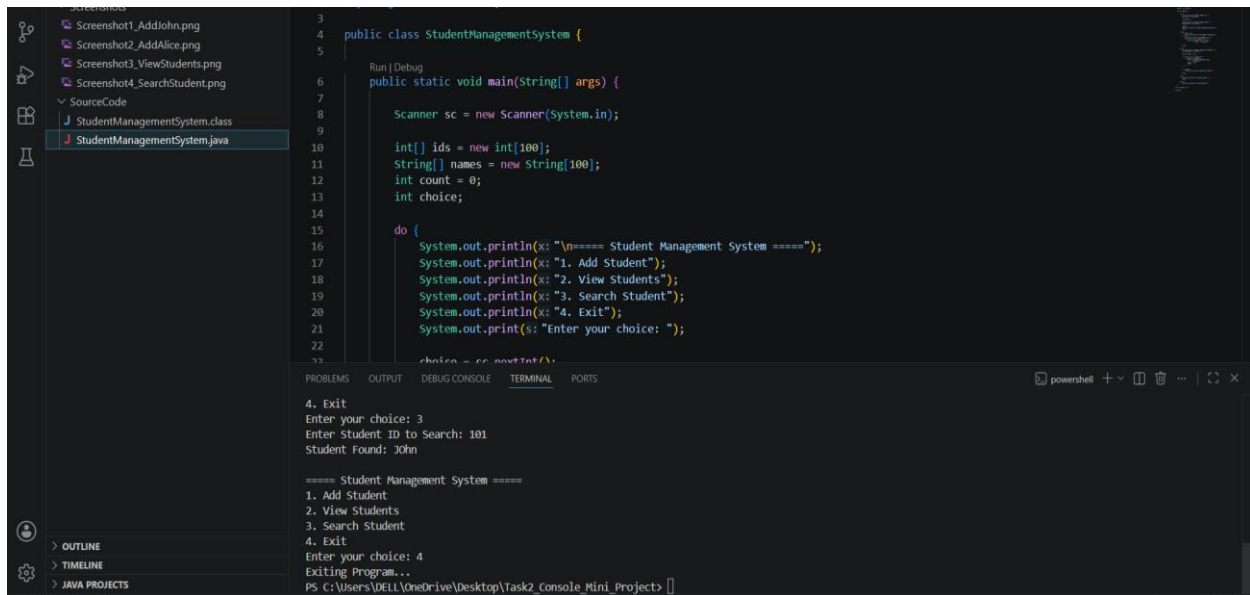
```
3
4 public class StudentManagementSystem {
5
6     Run | Debug
7     public static void main(String[] args) {
8
9         Scanner sc = new Scanner(System.in);
10
11         int[] ids = new int[100];
12         String[] names = new String[100];
13         int count = 0;
14         int choice;
15
16         do {
17             System.out.println(x: "\n==== Student Management System =====");
18             System.out.println(x: "1. Add Student");
19             System.out.println(x: "2. View Students");
20             System.out.println(x: "3. Search Student");
21             System.out.println(x: "4. Exit");
22             System.out.print(s: "Enter your choice: ");
23
24             choice = sc.nextInt();
25
26         } while (choice != 0);
27
28     }
29 }
```

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
ID: 101 Name: John
ID: 102 Name: Alice

===== Student Management System =====
1. Add Student
2. View Students
3. Search Student
Enter your choice: 3
Enter Student ID to Search: 101
Student Found: John
```

Figure 9: Searching a Student Record

Explanation: This output demonstrates the Search Student functionality of the Student Management System. The user enters the Student ID (101), and the system successfully retrieves and displays the corresponding student record (John), confirming that the search operation works correctly.



The screenshot shows an IDE with a project named 'Task2_Console_Mini_Project'. The file 'StudentManagementSystem.java' is open, displaying the following code:

```
3  
4 public class StudentManagementSystem {  
5  
6     public static void main(String[] args) {  
7  
8         Scanner sc = new Scanner(System.in);  
9  
10        int[] ids = new int[100];  
11        String[] names = new String[100];  
12        int count = 0;  
13        int choice;  
14  
15        do {  
16            System.out.println(x: "\n==== Student Management System =====");  
17            System.out.println(x: "1. Add Student");  
18            System.out.println(x: "2. View Students");  
19            System.out.println(x: "3. Search Student");  
20            System.out.println(x: "4. Exit");  
21            System.out.print(x: "Enter your choice: ");  
22  
23            choice = sc.nextInt();  
24  
25            switch(choice) {  
26                case 1: addStudent(sc);  
27                case 2: viewStudents();  
28                case 3: searchStudent(sc);  
29                case 4: exit();  
30            }  
31        } while (choice != 4);  
32    }  
33  
34    private void addStudent(Scanner sc) {  
35        if (count == 0) {  
36            System.out.println(x: "No students found. Adding a new student.");  
37        }  
38        System.out.println(x: "Enter student ID: ");  
39        int id = sc.nextInt();  
40        System.out.println(x: "Enter student name: ");  
41        String name = sc.next();  
42        ids[id] = id;  
43        names[id] = name;  
44        count++;  
45        System.out.println(x: "Student added successfully.");  
46    }  
47  
48    private void viewStudents() {  
49        if (count == 0) {  
50            System.out.println(x: "No students found.");  
51        }  
52        System.out.println(x: "Viewing students...");  
53        for (int i = 0; i < count; i++) {  
54            System.out.println(x: "ID: " + ids[i] + " Name: " + names[i]);  
55        }  
56    }  
57  
58    private void searchStudent(Scanner sc) {  
59        System.out.println(x: "Enter student ID to search: ");  
60        int id = sc.nextInt();  
61        for (int i = 0; i < count; i++) {  
62            if (ids[i] == id) {  
63                System.out.println(x: "Student found: " + names[i]);  
64            }  
65        }  
66    }  
67  
68    private void exit() {  
69        System.out.println(x: "Exiting Program...");  
70    }  
71}
```

The terminal output shows the program's execution:

```
4. Exit  
Enter your choice: 3  
Enter Student ID to Search: 101  
Student Found: John  
  
==== Student Management System =====  
1. Add Student  
2. View Students  
3. Search Student  
4. Exit  
Enter your choice: 4  
Exiting Program...  
PS C:\Users\DELL\OneDrive\Desktop\Task2_Console_Mini_Project>
```

Figure 10: Exiting the Student Management System

Explanation: This output demonstrates the Exit functionality of the Student Management System. When the user selects option 4, the program displays the message "Exiting Program..." and terminates successfully.

10. LEARNING OUTCOMES

Through this project, the following learning outcomes were achieved:

- Gained practical experience in developing a console-based Java application.
- Understood the use of the Scanner class for accepting user input.
- Learned to use arrays for storing and managing data.
- Implemented loops, conditional statements, and switch-case statements effectively.
- Developed a menu-driven application using core Java concepts.
- Improved problem-solving, logical thinking, and debugging skills.
- Understood how fundamental Java concepts can be integrated to build a simple management system.

11. CONCLUSION

The Student Management System successfully fulfills the requirements of a Console-Based Mini Project by implementing fundamental Java programming concepts in a practical application. The project provides an efficient menu-driven interface through which users can add student information, view all stored records, search for specific student details, and exit the application safely.

Throughout the development of this project, important concepts such as arrays, loops, conditional statements, switch statements, and user input handling using the Scanner class were effectively applied. The project demonstrates how basic programming concepts can be integrated to create a functional and user-friendly management system.

Developing this application significantly improved programming proficiency, logical thinking, and problem-solving skills while providing valuable hands-on experience in building console-based applications using Java. The project serves as a strong foundation for developing more advanced Java applications involving databases, file handling, and graphical user interfaces in the future.

12. REFERENCES

- Oracle Java Documentation.
 - Oracle Java Tutorials.
 - Visual Studio Code Documentation.
-